

Data Science & AI Interview Questions — Set 2

21 Questions with Answers | ML, Deep Learning, NLP, Systems

1 Area Under the ROC Curve

Q What does AUC-ROC measure, how is it computed, and when is it not the right metric?

:

A AUC-ROC measures a classifier's ability to discriminate between positive and negative classes across all decision thresholds. The ROC curve plots TPR (recall) vs FPR at each threshold; AUC is the area under it (0.5 = random, 1.0 = perfect). Computed via the trapezoidal rule or equivalently: $P(\text{score}(\text{positive}) > \text{score}(\text{negative}))$. When NOT to use it: (1) Severe class imbalance — AUC can look high even with many false positives; use PR-AUC instead. (2) When misclassification costs differ (fraud, medical diagnosis) — optimize for F1 or recall at fixed precision. (3) When calibrated probabilities matter — AUC is rank-order only; use Brier score or log-loss for calibration.

2 Linear Regression Parameters

Q How are the parameters of a linear regression model estimated, and what assumptions must hold?

:

A Ordinary Least Squares (OLS) minimizes the sum of squared residuals: $\beta = (X^T X)^{-1} X^T y$ — closed-form solution. For large datasets, gradient descent is used instead. Key assumptions (LINE): (1) Linearity — relationship between X and y is linear. (2) Independence — observations are independent. (3) Normality — residuals are normally distributed (needed for inference, not prediction). (4) Equal variance (homoscedasticity) — residual variance is constant across X values. Violations: multicollinearity inflates standard errors (check VIF); heteroscedasticity makes SEs unreliable (use robust SEs); non-linearity → add polynomial terms or use a non-linear model. Regularization (Ridge/Lasso) handles multicollinearity and overfitting.

3 PCA and K-Means

Q How would you combine PCA and K-Means clustering, and what are the benefits and risks?

:

A Workflow: apply PCA first to reduce dimensionality, then run K-Means on the principal components. Benefits: (1) Removes noise — low-variance components (noise) are discarded, making clusters cleaner. (2) Solves the curse of dimensionality — K-Means distance metrics degrade in high dimensions. (3) Faster computation — fewer features. (4) Removes multicollinearity — PCA components are orthogonal, which K-Means handles well. Risks: (1) Information loss — dropped components may contain cluster-relevant signal. (2) Interpretability — clusters are in PC space, harder to explain. (3) PCA assumes linearity — use UMAP/t-SNE for non-linear structure before clustering. Choose number of PCs by retaining 90-95% of variance; choose k via elbow method or silhouette score.

4 Score Based on Review

Q How would you build a system to generate a quality score for a product based on its user reviews?
:

A Multi-signal approach: (1) Sentiment analysis: fine-tuned BERT or VADER on review text → sentiment score per review. (2) Aspect-based sentiment: extract scores per aspect (battery life, screen quality) using aspect-sentiment models. (3) Rating signal: star rating, but weight by reviewer credibility (review count, helpful votes). (4) Recency weighting: exponential decay — recent reviews matter more. (5) Volume adjustment: Wilson score confidence interval to penalize products with very few reviews (avoids 5-star from 1 review). Aggregate: weighted average of sentiment scores, normalized to 0-100. Validate against human quality rankings. Detect and down-weight fake reviews (cluster analysis of reviewer behavior, posting patterns).

5 Bomb Detection

Q You are designing an ML system to detect bombs in airport X-ray scans. What model would you use and how would you handle the constraints?
:

A Frame as object detection (locate + classify threat objects in X-ray images). Model: fine-tuned YOLO or Faster R-CNN on annotated X-ray threat datasets. Key constraints: (1) Recall must be maximized — a missed bomb is catastrophic; accept high false positive rate (screeners review flagged bags). Set threshold extremely low. (2) Class imbalance — real threat images are rare; use data augmentation (rotate, scale, synthetic threat insertion), transfer learning from pretrained vision models. (3) Explainability — highlight bounding box of detected object for screener. (4) Speed — must process bags in real time (<2s). (5) Adversarial robustness — test against concealment strategies. Validate on held-out threat scenarios, track false negative rate as primary KPI.

6 Generative vs Discriminative

Q What is the difference between generative and discriminative models? Give examples of each.
:

A Discriminative models: learn $P(y|X)$ directly — the decision boundary between classes. Examples: logistic regression, SVM, neural networks, random forests. They focus only on classification and tend to perform better with sufficient labeled data. Generative models: learn the joint distribution $P(X, y) = P(X|y) * P(y)$ — they model how data is generated per class. At inference, use Bayes' theorem: $P(y|X) \propto P(X|y) * P(y)$. Examples: Naive Bayes, LDA, GMMs, VAEs, GANs. Generative models can generate new samples, handle missing data via the joint distribution, and work better with limited labeled data. Trade-off: discriminative models are usually more accurate for classification; generative models are more flexible and interpretable about data structure.

7 Using APIs for Downstream Tasks

Q You are an ML engineer at a bank with access to Reddit finance/news APIs and Bloomberg stock price API.
: Design an ML system that extracts, transforms, and stores data for downstream modeling teams.

A Pipeline: (1) Ingestion: scheduled jobs (Airflow) pull Reddit posts (PRAW) and Bloomberg prices via APIs. Stream Reddit in near-real-time; Bloomberg daily batch. (2) Raw storage: land raw JSON/CSV in a data lake (S3) with date partitioning. (3) Transformation: NLP on Reddit — sentiment scoring, entity extraction (ticker mentions via NER), topic classification. Bloomberg — compute derived features: returns, volatility, moving averages, RSI. (4) Feature store: store processed features in a feature store (Feast/Tecton) keyed by (ticker, date) so any downstream team can retrieve consistent features. (5) Schema: versioned feature definitions with documentation. (6) Quality: data validation (Great Expectations), freshness monitoring, anomaly detection on feature distributions. Downstream teams query the feature store, not raw APIs.

8 ReLU vs Tanh

Q Compare ReLU and Tanh activation functions. When would you prefer one over the other?
:

A Tanh: output in $(-1, 1)$, zero-centered (helps gradient flow in early layers), but suffers from vanishing gradients for large $|x|$ — saturates at ± 1 . ReLU: output = $\max(0, x)$. No saturation for positive values, sparse activations (many neurons output 0 = regularizing effect), computationally cheap. Preferred in most deep networks. Dying ReLU problem: neurons can get stuck at 0 if weights push inputs consistently negative — fix with Leaky ReLU (small negative slope) or ELU. Prefer Tanh: RNNs/LSTMs (bounded output prevents exploding activations), shallow networks, output layers for normalized targets. Prefer ReLU (or variants): deep networks (CNNs, transformers), hidden layers where speed and avoidance of vanishing gradients matter. GELU is now preferred in transformers (smoother than ReLU).

9 TikTok FYP Algorithm

Q How would you design the TikTok For You Page (FYP) recommendation algorithm?
:

A Two-stage architecture: (1) Candidate generation: retrieve ~1000 candidate videos from billions using lightweight models — collaborative filtering (user-video interaction matrix), content embeddings (video topics, audio, text), social graph (who you follow). (2) Ranking: score the ~1000 candidates with a heavy ranking model — multi-task neural network predicting multiple objectives: $P(\text{watch to completion})$, $P(\text{like})$, $P(\text{share})$, $P(\text{follow creator})$, $P(\text{not interested})$. Final score = weighted combination of these signals. (3) Re-ranking: apply diversity (avoid 10 consecutive cat videos), freshness boost, creator equity (surface new creators), safety filters. Features: video embedding (visual + audio + text), user history (recent watches, interactions), contextual (time of day, device, session length). Train with logged implicit feedback; evaluate with watch time and retention, not just CTR.

10 Intelligent Restaurant Review

Q How would you build an NLP system that intelligently summarizes and extracts insights from restaurant reviews?

A Pipeline: (1) Aspect extraction: identify aspects mentioned (food, service, ambiance, price, wait time) using span extraction (fine-tuned NER or seq-labeling model). (2) Sentiment per aspect: classify sentiment (positive/negative/neutral) for each aspect span. (3) Summarization: use an abstractive summarization model (BART, Pegasus, or prompt GPT) to generate a concise summary per aspect category from top-k most helpful reviews. (4) Highlights: extract representative quotes per aspect with highest confidence scores. (5) Aggregate scores: weighted sentiment score per aspect, displayed as a radar chart. (6) Trend detection: compare aspect scores over time to flag improvement/decline. Evaluation: ROUGE for summarization quality, human evaluation for accuracy of aspect attribution. Update weekly as new reviews arrive.

11 Correlated Features in Forests

Q How does feature correlation affect Random Forests, and how does it differ from its effect on linear models?

A In Random Forests, feature correlation reduces diversity between trees — when two features are highly correlated, they compete for splits, and the one randomly selected more often gets more importance credit while the other appears less important (importance split between them). This can mislead feature importance interpretation but doesn't badly hurt predictive performance. Fix: use permutation importance instead of Gini importance, as it is less biased. In linear models (linear regression), multicollinearity inflates standard errors of correlated coefficients — estimates become unstable and interpretation breaks down. Fix: Ridge regression, VIF analysis, PCA. Key difference: RF is robust to multicollinearity for prediction but not for feature importance; linear models are fragile for both prediction and interpretation.

12 Automated Labeling

Q You have 1 million unlabeled text documents. How would you build an automated labeling system?

A Approaches: (1) Zero-shot classification: use a pretrained NLI model (e.g., BART-large-mnli) to classify documents against label descriptions — no training data needed. Good starting point. (2) Weak supervision (Snorkel): define labeling functions (keyword rules, regex, distant supervision from knowledge bases) → train a label model that denoises and aggregates conflicting signals → generate probabilistic labels. (3) Active learning: label a small seed set → train a model → query the model for its most uncertain predictions → human labels those → retrain. Maximizes label efficiency. (4) Self-training: label seed set → train model → pseudo-label high-confidence unlabeled examples → retrain. Evaluate by having humans label a held-out sample and measuring agreement. Combine: weak supervision for initial labels + active learning to refine.

13 Explain Neural Nets to Kids

Q How would you explain how a neural network learns to a child?
:

A Imagine you're learning to recognize cats. You look at thousands of pictures, and each time someone tells you 'yes, cat' or 'no, not a cat.' Your brain adjusts — you notice cats have pointy ears, whiskers, fur. A neural network does the same thing with math. It has layers of tiny switches (neurons). Each switch looks at the picture and decides: 'does this part look cat-like?' The first layer notices simple things like edges and colors. The next layer combines those into shapes like ears and eyes. The last layer decides: cat or not? When it gets it wrong, it learns from its mistake — like when you get a quiz question wrong and remember the right answer next time. After seeing millions of pictures and correcting millions of mistakes, it gets really good at recognizing cats. That learning-from-mistakes process is called backpropagation.

14 Listings Recommendation

Q How would you build a listings recommendation system for a real estate platform (like Zillow)?
:

A Two-stage system: (1) Candidate retrieval: filter by user's hard constraints (price range, bedrooms, location radius) from a structured DB. Augment with embedding-based retrieval — encode listing features (price, sqft, style, amenities) and user preference history into a shared embedding space (two-tower model); retrieve nearest listings. (2) Ranking: score candidates with a gradient-boosted ranker trained on implicit signals (clicks, saves, inquiry submitted, tour scheduled). Features: listing-user affinity (embedding similarity), days on market, price change, match score on stated preferences, commute time to user's workplace. (3) Diversity: ensure varied neighborhoods and price points. (4) Cold start: for new users, use stated preferences from onboarding. Evaluate offline with NDCG; online with inquiry rate and tour conversion via A/B test.

15 Measuring Text Difficulty

Q How would you build a model to measure the reading difficulty level of a text passage?
:

A Feature-based approach: (1) Traditional readability formulas: Flesch-Kincaid Grade Level (uses sentence length + syllable count), Gunning Fog Index, SMOG. Fast and interpretable. (2) Linguistic features: average word frequency (rare words = harder), sentence length distribution, syntactic complexity (parse tree depth), proportion of subordinate clauses, lexical diversity (TTR), proportion of academic vocabulary (AWL). (3) ML approach: train a regression or ordinal classifier on labeled corpora (e.g., CommonLit, WeeBit — texts graded by reading level). Features: all of the above + TF-IDF + sentence embeddings. (4) LLM approach: prompt a language model to estimate grade level with chain-of-thought. Evaluate with MAE vs human-assigned grade levels on a held-out benchmark.

16 Tradeoffs Between Precision and Recall

Q Explain the precision-recall tradeoff and how you decide where to set the threshold.
:

A Precision = $TP / (TP + FP)$: of all predicted positives, how many are truly positive. Recall = $TP / (TP + FN)$: of all actual positives, how many did we catch. They trade off via the decision threshold: lowering threshold $\rightarrow$ more positives predicted $\rightarrow$ recall increases, precision decreases (more FPs). Raising threshold $\rightarrow$ fewer positives $\rightarrow$ precision increases, recall decreases (more FNs). How to choose threshold: (1) Business cost of FP vs FN — in spam detection, FN (missed spam) is annoying; FP (blocking real email) is worse $\rightarrow$ tune for high precision. In cancer screening, FN (missed cancer) is catastrophic $\rightarrow$ maximize recall. (2) F1 score: harmonic mean of P and R — good when costs are balanced. (3) F-beta: beta > 1 weights recall more; beta < 1 weights precision more. Plot PR curve and choose threshold at the business-optimal operating point.

17 Inception Architecture

Q Explain the Inception (GoogLeNet) architecture and the motivation behind Inception modules.
:

A Inception modules apply multiple filter sizes (1x1, 3x3, 5x5) and max pooling in parallel on the same input, then concatenate the outputs along the channel dimension. Motivation: objects appear at different scales in images — a small filter captures fine details, a large filter captures broader context. Rather than choosing one, Inception learns what combination works best. 1x1 convolutions (bottleneck layers) are applied before 3x3 and 5x5 filters to reduce channel depth, dramatically cutting computational cost. Key innovations: (1) Multi-scale feature extraction in parallel. (2) Computational efficiency via 1x1 bottleneck. (3) Auxiliary classifiers at intermediate layers during training to combat vanishing gradients (removed in later versions). InceptionV3 and InceptionResNet added batch normalization and residual connections. Inception demonstrated that going 'wider' can be as effective as going deeper.

18 Meta-classifier in Stacking

Q How does stacking work in ensemble learning, and what is the role of the meta-classifier?
:

A Stacking (stacked generalization): train multiple diverse base models (level-0: e.g., LightGBM, Random Forest, SVM, logistic regression) on training data. Generate out-of-fold predictions from each base model using cross-validation — this prevents leakage. Stack these predictions as new features $\rightarrow$ train a meta-classifier (level-1) to learn how to best combine the base model outputs into a final prediction. Meta-classifier role: it learns which base models are most reliable in which regions of the feature space — it can downweight a model that systematically fails on certain examples. Typical meta-classifiers: logistic regression (simple, prevents overfitting), LightGBM (more flexible). Key: base models should be diverse (different algorithms, feature subsets) to give the meta-learner complementary signals. Often yields 1-3% improvement over the best single model.

19

Chronological Order in Boosting

Q Why is it important to respect chronological order when training boosting models on time-series data?
:

A Boosting (and ML generally) assumes training and test data are i.i.d. — but time-series data has temporal dependencies. If you randomly shuffle and split, future data leaks into training: the model learns patterns from the 'future' that it wouldn't have in production, giving an overly optimistic evaluation metric. In practice this means: (1) Always use a temporal train/validation/test split — train on data up to time T , validate on T to $T+k$, test on $T+k$ onward. (2) Lag features must only use past values ($t-1$, $t-7$, etc.) — never current or future values. (3) Rolling-window cross-validation: train on expanding or sliding windows, always predicting the next period. (4) Features like 'day of week' or 'month' are fine; 'next week's sales' is leakage. Violating this leads to models that look great offline but fail in production.

20

Backpropagation Explanation

Q Explain how backpropagation works in a neural network.
:

A Backpropagation is the algorithm for computing gradients of the loss with respect to every parameter in the network, using the chain rule of calculus. Forward pass: input flows through layers, applying weights and activations, producing a prediction. Compute loss (e.g., cross-entropy). Backward pass: starting from the loss, compute $dL/d(\text{output})$, then propagate backward layer by layer. At each layer: $dL/dW = dL/d(\text{output}) * d(\text{output})/dW$ using the chain rule. Gradients accumulate for shared weights. Parameter update: optimizer (SGD, Adam) uses gradients to update weights: $W = W - lr * dL/dW$. Key insight: the chain rule allows efficient reuse of intermediate gradient computations (dynamic programming). Vanishing gradients occur when chain-multiplied derivatives shrink to near zero in deep networks — solved by ReLU, batch norm, residual connections.

21

Email Marketing System

Q How would you design an ML system to optimize email marketing campaigns?
:

A System components: (1) Send-time optimization: predict the time each user is most likely to open email — train a model on historical open timestamps per user. (2) Subject line optimization: A/B test or multi-armed bandit (Thompson Sampling) across subject line variants; predict open rate using NLP features of subject text. (3) Content personalization: recommend content blocks (products, articles) based on user preferences — collaborative filtering + user segments. (4) Audience segmentation: cluster users by engagement level (champions, at-risk, dormant) → tailor frequency and content per segment. (5) Churn/unsubscribe prediction: identify users at risk of unsubscribing → reduce frequency or change content. (6) Holdout group: always maintain a pure control group to measure true campaign lift. Evaluate: open rate, CTR, conversion rate, revenue per email, unsubscribe rate. Run proper A/B tests for each optimization lever.